VELOZITY GLOBAL SOLUTIONS
Technical Hiring Assessment

**Task Role:** Full Stack Developer | **Deadline:** 25/03/2026, 11:59 PM IST 11:59 PM IST |

**AI Tools:** Strictly Prohibited

---

**Task: Build a Real-Time Client Project Dashboard with Role-Based Access & Live Activity Feed**

**Overview** Build a full stack web application that a small agency uses internally to manage client projects, track task progress, and monitor team activity in real time. This is not a tutorial CRUD app — it requires WebSocket implementation, role-based permission logic, and state management decisions that require genuine engineering judgment.

**What You Must Build**

**1. Authentication & Role System**

Three roles with strictly different access levels:

| Role | Access |
|---|---|
| Admin | Full access — manage clients, projects, users, view all activity |
| Project Manager | Create & manage projects, assign tasks, view their team's activity only |
| Developer | View assigned tasks only, update task status, cannot see other developers' tasks |

- JWT-based authentication (access token + refresh token)
- Refresh token must be stored in an HttpOnly cookie — not localStorage
- Role middleware must be enforced on every protected route at the API level — frontend-only role hiding is not acceptable
- A Developer must not be able to reach a Project Manager's data even by directly hitting the API endpoint with a modified token

**2. Project & Task Management**

- Admin and PM can create projects and assign them to clients

- Projects contain tasks — each task has: title, description, assigned developer, status (To Do / In Progress / In Review / Done), priority (Low / Medium / High / Critical), due date, and an activity log
- Task status changes must be recorded with a timestamp and the user who made the change — this log must be stored in the database, not derived
- PM can only manage projects they created — they cannot see or edit another PM's projects
- Tasks past their due date must be automatically flagged as Overdue — this must happen via a scheduled background job, not on page load

**3. Real-Time Activity Feed**

This is the core technical challenge of the task.

- Implement a live activity feed using WebSockets (Socket.io or native WebSocket — your choice, justify it in the README)
- When any user updates a task status, all users currently viewing that project must see the update in real time without refreshing
- The feed must show: who made the change, what they changed, and when — formatted as "Ravi moved Task #12 from In Progress → In Review · 2 mins ago"
- Admin sees activity across all projects in a single global feed
- PM sees activity only from their own projects
- Developer sees activity only on tasks assigned to them
- If a user is offline and comes back, they must see the last 20 activity events they missed — this must be fetched from the database, not cached in memory

**4. Dashboard & Filters**

- Admin dashboard: total projects, total tasks by status, overdue task count, active users online right now (shown as a live count using WebSocket presence)
- PM dashboard: their projects summary, tasks by priority, upcoming due dates this week
- Developer dashboard: their assigned tasks, sorted by priority then due date
- All task lists must support filtering by status, priority, and due date range — filters must work via query parameters so they are shareable as URLs

**5. Notifications**

- When a task is assigned to a developer, they receive an in-app notification (stored in DB, shown in UI)
- When a task they own is moved to In Review, the PM receives a notification
- Notifications must show as a count badge and expand into a dropdown — mark as read individually or all at once
- Unread notification count must update in real time via WebSocket, not polling

**Technical Requirements**

- **Frontend:** React with TypeScript — no plain JavaScript accepted
- **Backend:** Node.js with Express or Fastify — your choice, justify it
- **Database:** PostgreSQL — use proper relational schema with foreign keys, indexes on frequently queried columns, and explain your indexing decisions in the README
- **ORM:** Prisma or raw SQL — no MongoDB, no NoSQL
- **Real-time:** WebSocket — no long-polling, no SSE
- **Background Jobs:** Use node-cron or Bull queue for the overdue task scheduler — justify your choice
- **Validation:** All API inputs must be validated server-side — frontend validation alone is not sufficient
- **Error Handling:** All API endpoints must return consistent, structured error responses — no raw stack traces exposed to the client
- **Environment:** All secrets in .env — never hardcoded

**Seed Data Required** Your repository must include a seed script that creates:

- 1 Admin, 2 Project Managers, 4 Developers
- At least 3 projects with 5+ tasks each in various statuses
- At least 2 tasks already in overdue state
- Pre-existing activity log entries so the feed is not empty on first load

**Submission Requirements**

- Public GitHub/GitLab repository
- README must include: local setup instructions (Docker preferred), database schema diagram or description, architectural decisions (WebSocket library choice, job queue choice, token storage approach), known limitations
- In the Explanation field (150–250 words): the hardest problem you solved, how you handled the real-time role-filtered feed, and one thing you'd do differently

**Evaluation**

| Criteria | Weight |
|---|---|
| Role-based access — enforced at API level, not just frontend | 25% |
| Real-time feed — correct, role-filtered, with missed event catchup | 25% |

| | |
|---|---|
| Database design — schema quality, relationships, indexing | 20% |
| Code architecture — separation of concerns, TypeScript usage | 20% |
| Seed data, README, setup experience | 10% |

**Auto-Disqualification:** Role access enforced only on frontend, WebSocket replaced with polling, no seed script, no TypeScript, raw SQL mixed randomly into controllers, hardcoded secrets, missing refresh token implementation.

**Completed the task? Submit your work here:** (https://bit.ly/4bGXmZV)